VELTRIS
INNOVATE | ACCELERATE | TRANSFORM

PROJECT REPORT · SMALL LANGUAGE MODEL (SLM)

ComplaintIQ

A locally run, fine-tuned small language model that classifies financial consumer complaints by product and issue, with human review, routing and a full audit trail.

PREPARED FOR
Veltris

REPORT DATE
09 October 2026

DEPLOYED MODEL
Qwen3-4B-Instruct-2507, QLoRA fine-tuned, Q4_K_M GGUF

RUNS ON
One Windows 11 PC with an NVIDIA RTX 4070 · no cloud service at inference

1 Executive summary

ComplaintIQ reads a customer's complaint narrative and predicts which financial product it concerns and what the issue is, choosing from nine supported product/issue pairs, or flags it as out of scope. A reviewer confirms or corrects every prediction in a desktop web application; nothing is sent to an external AI service.

90.3%	64.5%	204 ms	2.0 h
product accuracy	**10-class accuracy**	**mean response time**	**training time**
3,000 newest complaints (Feb-Jul 2026)	same 3,000 complaints, incl. out of scope	in the app, RTX 4070	950 QLoRA steps on one RTX 4070

What was built

- **Two fine-tuning rounds.** Phase 1 fine-tuned Qwen2.5-1.5B-Instruct on 9,000 historical complaints in a cloud notebook and ran it on a MacBook Air M1. Phase 2 fine-tuned the larger **Qwen3-4B-Instruct-2507** on a 70,162-complaint, time-split dataset entirely on a local Windows PC, and it is now the deployed model.
- **A complete review application**: React/TypeScript interface, FastAPI service, SQLite case store with append-only history, durable processing queue, CSV import, backups and an evaluation page, all bound to `127.0.0.1`.
- **Measured evidence for every claim**: each figure in this report is read from a saved evaluation or log file in the project.

Key results

- On the project's original 153 reserved test complaints, the deployed Qwen3 model reaches **90.85%** product and **58.82%** joint product/issue accuracy, against 88.89% / 54.25% for the Phase 1 Qwen2.5 model and 88.24% / 55.56% for the TF-IDF baseline, and answers about 4.7x faster than Phase 1 did on the Mac (204 ms vs 953 ms, different hardware).
- The model never produced an invalid label (**0** invalid replies in 3,000 test cases), because decoding is constrained to the allowed answers.
- Out-of-scope detection: precision 82.3% and recall 65.3% on the test set; 88.9% of hand-written unrelated texts were flagged.

Honest limits

- The three credit-reporting issues are the hardest classes (F1 about 48-50%): their narratives overlap heavily.
- Compressing the model to 4-bit GGUF for fast local serving cost accuracy: on 300 identical test rows the GGUF scored 59.7% vs 64.7% for the unquantized fine-tuned model.
- To fit a three-hour window, training stopped after 950 steps (about 22% of one epoch). A full epoch may score differently.

2 Problem, scope and approach

Financial institutions receive large volumes of free-text complaints. Routing each one to the right team depends on understanding which product it concerns and what went wrong. The assignment was to build a working complaint-intelligence pilot around a **small language model that is fine-tuned on the task and runs locally**, measured honestly against a traditional baseline.

Label space

Three product families and nine issue pairs, chosen from the public CFPB Consumer Complaint Database where narratives are plentiful, plus one out-of-scope class added in Phase 2 so the model itself can decline unrelated text.

#	Product	Issue
1	Checking or savings account	Deposits and withdrawals
2	Checking or savings account	Managing an account
3	Checking or savings account	Problems caused by my funds being low
4	Credit card	Billing disputes
5	Credit card	Fees or interest
6	Credit card	Other
7	Credit reporting	Improper use of your report
8	Credit reporting	Incorrect information on your report
9	Credit reporting	Problem with a credit reporting company's investigation into an existing problem
10	Unknown	Needs review

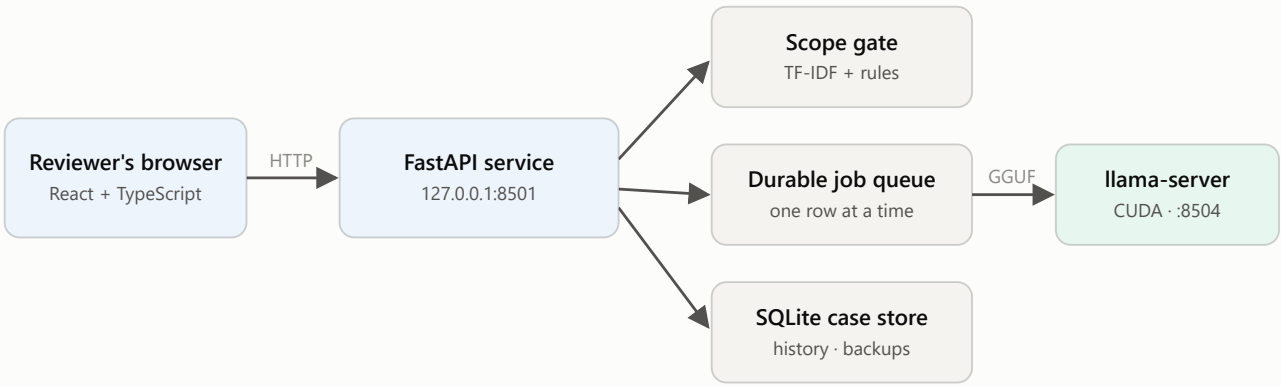
Approach

1. **Baseline first.** A TF-IDF + logistic-regression classifier sets the bar (product 88.24%, joint 55.56% on 153 reserved complaints).
2. **Choose a model that fits the hardware.** Candidates were benchmarked locally before any training (Section 6).
3. **Fine-tune with QLoRA** (4-bit base model + small trainable LoRA adapters), so training fits a single consumer GPU.
4. **Generative classifier with constrained decoding.** The model answers with the application's exact JSON (`{"product": ..., "issue": ...}`), and decoding is restricted to the ten valid answers, so every reply is a legal label.
5. **Export to 4-bit GGUF** and serve it with llama.cpp behind the application; a human confirms every decision.

> Priority, department routing and suggested next steps are explicit, documented rules, not model predictions. Summaries are extractive previews. No confidence or risk percentages are invented.

3 System architecture

Figure 1 · Runtime components (all on one machine)



Every process binds to the loopback interface only. The model server holds one Q4_K_M GGUF and handles one request at a time; failures surface as explicit errors, never a silent switch to the baseline.

Request flow

1. A complaint is submitted (single narrative or CSV of up to 100 rows / 2 MB). It is written to the durable queue before inference.
2. The **scope gate** checks for card, credit-report or bank-account context and runs a TF-IDF classifier trained on supported vs other-product complaints; out-of-scope text goes to review without calling the model.
3. The first 2,000 normalized characters go to **llama-server** with the system prompt and a JSON schema that only allows the valid product/issue pairs. The reply is validated again in Python.
4. Rules add priority, department and a suggested next step; the case is saved with its original prediction, which is never overwritten.
5. A reviewer confirms or corrects the labels (a reason is required), adds notes and moves the case through *Queued* → *In review* → *Resolved*. Every change is appended to the case history.

Components

Layer	Technology	Notes
Interface	React 18, TypeScript, Vite	Cases queue, case detail and review, analysis, CSV imports, model & system page
API	FastAPI, Uvicorn (Python 3.12)	Same-origin writes only, loopback hosts, revision checks reject stale edits (HTTP 409)
Storage	SQLite (versioned schema)	Cases, append-only events, jobs; daily automatic + manual backups; validated restore tool
Inference	llama.cpp b11496 llama-server (CUDA 12.4)	2,048-token context, 1 slot, JSON-schema constrained decoding
Model	complaintiq-qwen3-4b-Q4_K_M.gguf	Q4_K_M, 2.33 GiB, SHA-256 9a810438c1acac30...
Baseline	scikit-learn TF-IDF + logistic regression	Trained from CSV at start-up (no pickled models); LIME explanations

4 Training data

Both phases use real, public consumer complaint narratives from the U.S. Consumer Financial Protection Bureau (CFPB). Labels are the product and issue the consumer selected when filing; they are not independently adjudicated.

Phase 1 dataset (Qwen2.5-1.5B)

Property	Value
Source	Official CFPB narratives archive, December 2011 - April 2018 (1,031,863 records)
Selected complaint dates	19 March 2015 - 30 April 2018
Split sizes	9,000 training · 900 validation · 900 test (1,000 / 100 / 100 per issue)
Classes	9 product/issue pairs (no out-of-scope examples)
Leakage control	Zero shared IDs or input hashes across splits; 65 near-duplicates (word-shingle Jaccard ≥ 0.95) rejected
Original reserved test	153 complaints kept in the test split since the baseline was built; used for every cross-model comparison

Phase 2 dataset (Qwen3-4B, deployed)

Property	Value
Source	Official CFPB Consumer Complaint Database export (complaints.csv.zip) (archived 15 Aug 2026 capture; the live export no longer includes narratives)
Archive SHA-256	`d92239489715c816fa4fc77a...`
Eligible narratives scanned	3,510,009 (2015-03 to 2026-07)
Split policy	Time-based: newest months are the test set, so the model is tested on complaints newer than anything it trained on
Training	70,162 complaints (2015-03 to 2025-11), about 7,000 per class
Validation	1,938 complaints (2025-12 to 2026-01)
Test	3,000 complaints (2026-02 to 2026-07), 300 per class
Check set (held out)	1,407: the original 153 test complaints, 1,200 other-product complaints and 54 hand-written unrelated texts
Out-of-scope class	Other CFPB products (debt collection, mortgage, student loan, vehicle/consumer loan, payday/personal loan, money transfer, prepaid card, debt management) sampled evenly, plus 54 authored non-financial examples (x3)
De-duplication	Exact duplicates removed across splits (test wins over validation over train); training rows near-duplicating any evaluation row (shingle Jaccard ≥ 0.9) dropped: 3,052
Model input	Whitespace-normalized first 2,000 characters, identical to what the app sends
Token lengths (Qwen3 tokenizer)	mean 369, median 363, capped at 512; 17,877 of 70,162 training complaints (25%) shortened to fit 512 tokens

> CFPB issue names changed in 2017; older and newer names are mapped onto the nine pairs (e.g. "Problem with a purchase shown on your statement" → Billing disputes). The full mapping and file hashes are in `pc_kit/data/manifest.json`.

5 Devices and software

Device	Specification	Used for
Apple MacBook Air (M1)	Apple M1, 8 GB unified memory, macOS · llama.cpp with Metal	Phase 1: data preparation, baseline, local model benchmark, running the Qwen2.5 app; preparing the Windows training kit
Google Colab (cloud notebook)	Cloud GPU runtime · Unsloth	Phase 1: QLoRA training of Qwen2.5-1.5B and Q4_K_M export
Lenovo workstation (30FS)	Intel Core i7-14700K (20 cores / 28 threads) · 32 GB RAM (31.6 GB usable) · **NVIDIA GeForce RTX 4070, 12 GB** (driver 610.60, compute 8.9) · Windows 11 Pro · ~950 GB data drive (D:) for the project	Phase 2: training, evaluation, GGUF export and running the deployed application

Software (Phase 2, Windows PC)

Component	Version
Training environment	Python 3.11.9 · PyTorch 2.10.0 + CUDA 12.6 · Transformers 5.19.0 · PEFT 0.21.2 · bitsandbytes 0.50.2 · Accelerate 1.15.0
Application environment	Python 3.12.10 · FastAPI 0.142.2 · Uvicorn 0.54.0 · scikit-learn 1.9.1 · pandas 3.0.6
Frontend build	Node.js 24 · React 18 · TypeScript · Vite
Model serving / export	llama.cpp b11496 (llama-server and llama-quantize, CUDA 12.4 build; convert_hf_to_gguf)
Development tools	VS Code with Jupyter (training notebook), Claude Code (assistant), pytest, ruff, mypy, Vitest

Base models

Model	Parameters	Role
Qwen2.5-1.5B-Instruct	1.54 B	Phase 1 fine-tuned model (Mac)
Qwen3.5-0.8B	0.8 B	Benchmarked in Phase 1, not selected
Qwen3-4B-Instruct-2507	4.06 B (33.0 M trainable LoRA = 0.81%)	Phase 2 fine-tuned and deployed model

6 Phase 1: model selection and Qwen2.5-1.5B

Local model benchmark (before training, MacBook Air M1)

Two small instruction models were compared on the same 27 validation complaints (three per class) with identical prompts, before any fine-tuning. "Verified pair schema" means decoding was constrained to valid product/issue pairs.

Backend and decoding	Product acc.	Joint acc.	Valid output	Median latency
TF-IDF baseline	88.89%	55.56%	100%	0.8 ms
Qwen2.5-1.5B, free JSON	51.85%	25.93%	59.26%	1,005 ms
Qwen3.5-0.8B, free JSON	3.70%	0.00%	11.11%	755 ms
Qwen2.5-1.5B, verified pair schema	77.78%	48.15%	100%	944 ms
Qwen3.5-0.8B, verified pair schema	59.26%	22.22%	100%	646 ms

Source: docs/LOCAL_MODEL_COMPARISON.md and artifacts/local_*_validation.json. Constrained decoding was adopted for every later model.

Phase 1 training configuration (Google Colab, Unsloth)

Setting	Value
Base model	Qwen2.5-1.5B-Instruct, loaded in 4-bit
Method	QLoRA, LoRA rank 16, alpha 16, dropout 0
Batch	2 per step x 8 gradient accumulation = 16
Optimizer / schedule	Learning rate 2e-4, linear decay, 10 warm-up steps
Length	1 epoch over 9,000 complaints = 563 steps (exported checkpoint-563); max sequence 2,048 tokens
Export	Q4_K_M GGUF, 986 MB, served by llama.cpp with Metal on the M1

Phase 1 results

Configuration	Cases	Product acc.	Joint acc.
Original Qwen2.5, constrained (validation)	27	77.78%	48.15%
Fine-tuned Qwen2.5, constrained (validation)	27	96.30%	74.07%
TF-IDF baseline, reserved test	153	88.24%	55.56%
Fine-tuned Qwen2.5, reserved test	153	88.89%	54.25%

The large validation gain did not carry over to the reserved test, where Phase 1 matched but did not beat the baseline on joint accuracy. The model also had no way to decline unrelated text, so a separate scope gate was added. Both observations motivated Phase 2: a larger model, far more and newer training data, and an explicit out-of-scope class.

7 Phase 2: training Qwen3-4B on the PC

The training kit (scripts, data, pinned dependencies and a notebook) was prepared on the Mac and run on the Windows PC. Training ran from `pc_kit/notebooks/train_qwen3_windows.ipynb` in VS Code: environment check → dry run → speed test → training → evaluation → export.

Training configuration

Setting	Value
Base model	Qwen3-4B-Instruct-2507, 4-bit NF4 with double quantization, bf16 compute, SDPA attention
Adapters	LoRA rank 16, alpha 32, dropout 0.05, on all attention and MLP projections (33.0 M trainable parameters)
Batch	2 per step x 8 gradient accumulation = 16 sequences per optimizer step
Optimizer	AdamW, learning rate 0.0001, cosine schedule, 3% warm-up, gradient clipping 1.0, gradient checkpointing
Sequence length	512 tokens (prompt + answer); the complaint is shortened to fit, never the prompt; loss on answer tokens only
Length	950 steps = 15,200 complaints seen (22% of one 4,386-step epoch); 2.5 h safety stop (not reached)
Checkpoints	every 250 steps; checkpoint-950 and the final adapter are kept
Evaluation decoding	Greedy, constrained to the ten valid replies (token trie); prompts up to 1,024 tokens, matching the app's 2,000-character input
Seed	42

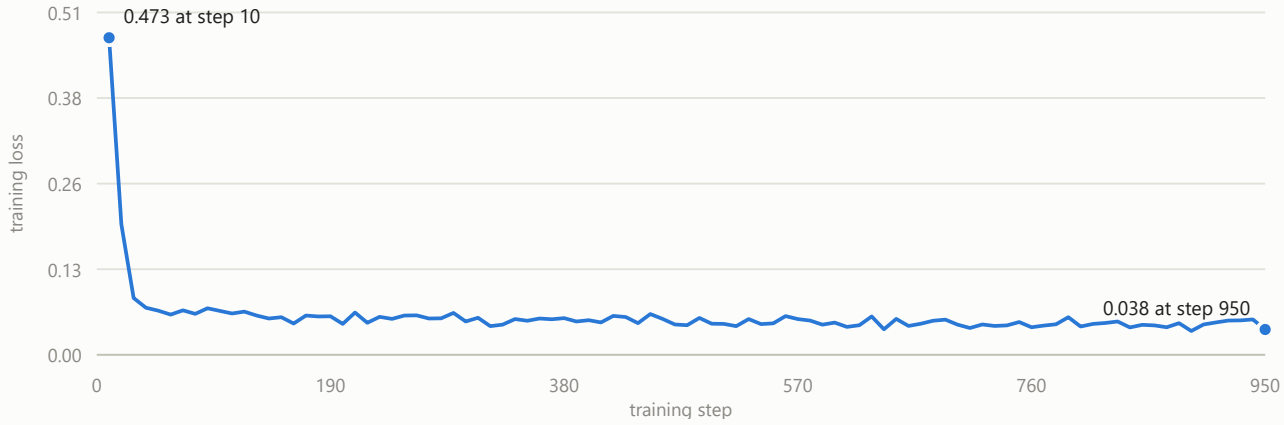
Making it fit the GPU

Configuration	Speed	GPU memory	Outcome
Batch 4 x 4 accumulation (kit default)	about 11-12 s/step (first steps of the 200-step speed test, batch 4 x 4; stopped)	11.9 of 12.3 GB in use (nvidia-smi); 100% busy but drawing only 121 of 200 W, i.e. waiting on memory spilled into shared system memory	Rejected: a full epoch would take about 14 h
Batch 2 x 8 accumulation (same effective batch 16)	7.10 s/step (30-step speed test, batch 2 x 8); 7.67 s/step over the full run	6.5 GB peak allocated by training steps; 10.3 GB in use (nvidia-smi) at 182 W, i.e. fully loaded without spilling	Used for the main run

Halving the per-step batch and doubling gradient accumulation keeps the optimisation identical (16 sequences per update) while stopping memory from spilling out of the 12 GB card. That made each step about a third faster. The step count was then chosen so that training plus evaluation and export would finish in about three hours.

Training progress

Figure 2 · Training loss over 950 steps (logged every 10 steps)



Loss on the answer tokens fell from 0.473 to 0.038; average training loss 0.0573. Cosine learning-rate schedule decays to zero at step 950. Source: pc_kit/results/qwen3-4b-main/train_log.jsonl.

Run timeline, 9 October 2026 (IST)

Time	Event
10:52	Automated setup started (RUN_ALL.bat); model and installers downloaded and SHA-256 verified
11:16-11:24	Setup stopped twice: PyTorch download failed its hash check (see Section 12)
~11:45	Verified PyTorch installed from a resumable download; packages installed from the notebook; environment check READY
11:50-12:05	Dry run passed (5 steps, adapter saved and reloaded); speed tests compared batch 4 x 4 with batch 2 x 8
12:08	Training started (950 steps)
14:11	Training finished: 2.02 h, final adapter saved
14:34	Automatic evaluation on 3,000 test + 1,407 check complaints finished
14:35-14:42	GGUF export: merge, convert, quantize to Q4_K_M and 300-row llama-server check
~14:45	Application started with the new model; end-to-end analysis verified in the browser

Export for serving

Step	Detail
Merge	Adapter merged into the bf16 base model on the CPU
Convert + quantize	llama.cpp b11496 convert_hf_to_gguf, then llama-quantize to Q4_K_M (2.33 GiB)
Identity	`complaintiq-qwen3-4b-Q4_K_M.gguf` · SHA-256 `9a810438c1acac3094aae0bf5c0f84a3...`
Regression check	300 fixed test rows sent through llama-server with the app's exact request

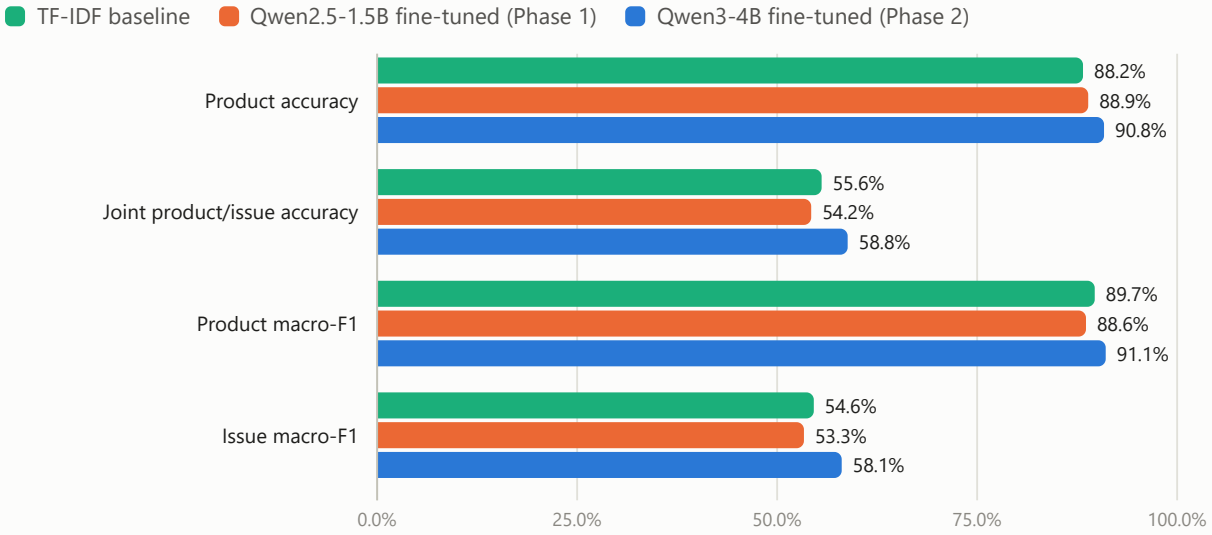
8 Benchmarks

Three benchmark sets are used, each answering a different question:

Set	Size	Question it answers
Original reserved test	153	How do all models compare on the same complaints? (2015-2018 data, nine in-scope classes)
2026 test set	3,000	How well does the deployed model handle the newest complaints, including out-of-scope ones? (10 classes, 300 each)
Check set	1,407	Does it decline other products and unrelated text while still classifying the 153 originals?

8.1 Across the project (153 reserved complaints, deployed form of each model)

Figure 3 · Accuracy and macro-F1 by model



Each model evaluated through its serving path with constrained decoding. Sources: artifacts/evaluation.json, finetuned_evaluation.json, qwen3_evaluation.json.

Model (153 reserved complaints)	Product acc.	Joint acc.	Product macro-F1	Issue macro-F1	Mean latency	Hardware
TF-IDF baseline	88.24%	55.56%	89.7%	54.6%	0.9 ms	CPU
Qwen2.5-1.5B fine-tuned	88.89%	54.25%	88.6%	53.3%	953 ms	M1, Metal
Qwen3-4B fine-tuned (GGUF)	90.85%	58.82%	91.1%	58.1%	204 ms	RTX 4070, CUDA

> Latency compares different machines (M1 laptop vs RTX 4070 desktop), so it reflects the whole deployment, not the model alone. Qwen3 latency: median 201 ms, 95th percentile 270 ms.

8.2 Deployed model on the newest complaints and the check set

Fine-tuned Qwen3-4B adapter, 4-bit base, greedy decoding constrained to the ten valid replies (pc_kit/src/evaluate_qwen3.py).

Metric	2026 test set (3,000)	Check set (1,407)
10-class accuracy	64.5%	74.5%
Macro-F1	65.0%	39.4%
Weighted F1	65.0%	80.8%
Product accuracy	90.3%	77.9%
In-scope joint product/issue accuracy	64.4%	58.2%
Out-of-scope precision	82.3%	99.6%
Out-of-scope recall	65.3%	76.5%
Invalid replies	0	0

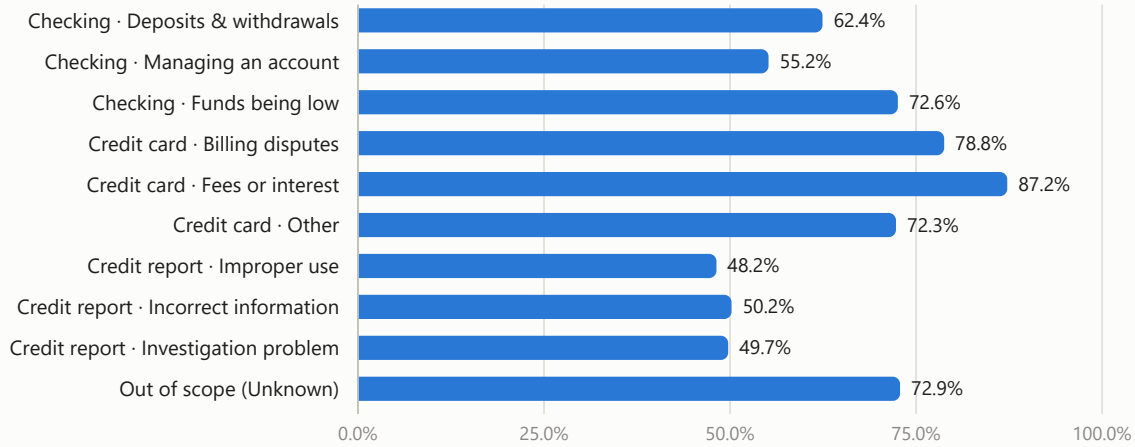
> The check set is dominated by out-of-scope complaints (1,254 of 1,407), so its macro-F1 is pulled down by nine in-scope classes with only 17 examples each; accuracy and weighted F1 describe it better.

Check set by source

Source	Complaints	Accuracy
Original 153 reserved test complaints	153	58.2%
Other-product CFPB complaints (should be flagged out of scope)	1200	75.9%
Hand-written unrelated texts (should be flagged out of scope)	54	88.9%

8.3 Per-class results (2026 test set, fine-tuned Qwen3-4B adapter)

Figure 4 · F1 score per class



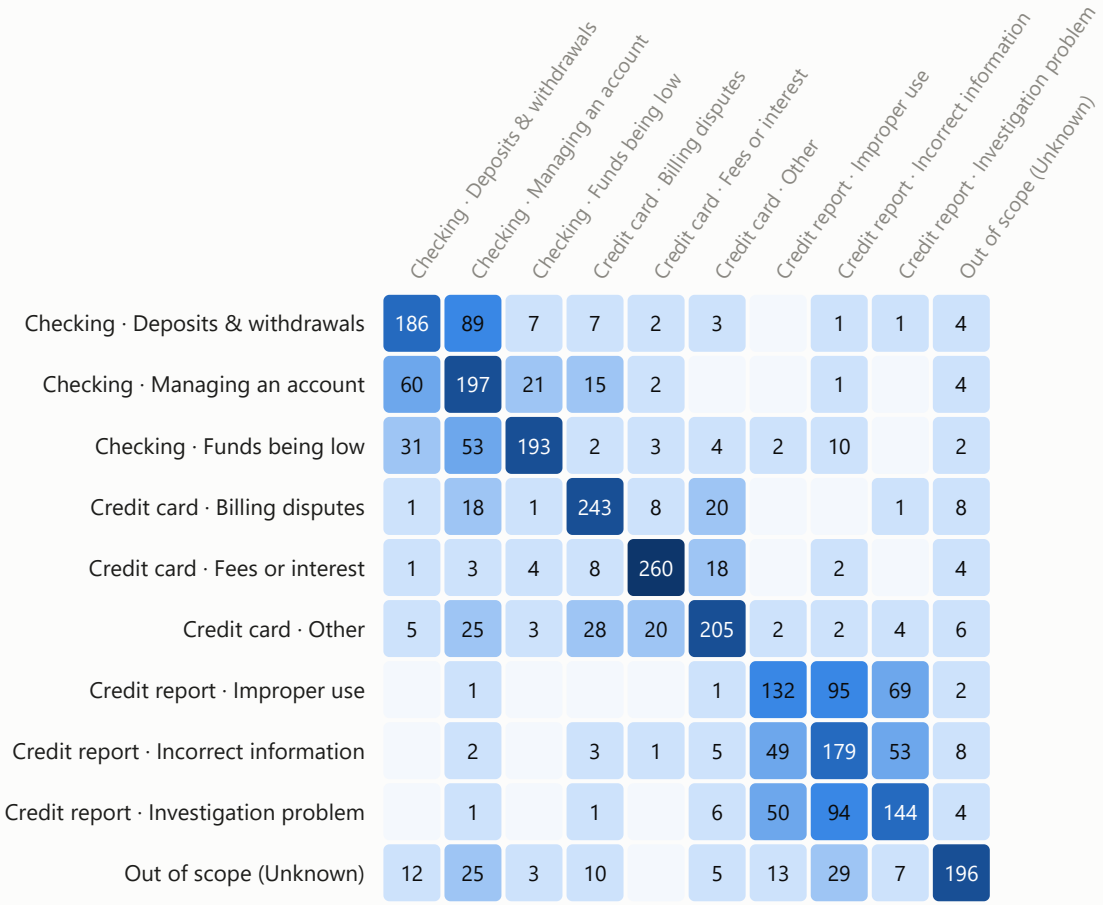
300 test complaints per class. Credit-reporting issues are hardest: complaints about incorrect information, improper use and the bureau's investigation often describe the same dispute.

Class	Precision	Recall	F1	Support
Checking · Deposits & withdrawals	62.8%	62.0%	62.4%	300
Checking · Managing an account	47.6%	65.7%	55.2%	300
Checking · Funds being low	83.2%	64.3%	72.6%	300
Credit card · Billing disputes	76.7%	81.0%	78.8%	300
Credit card · Fees or interest	87.8%	86.7%	87.2%	300
Credit card · Other	76.8%	68.3%	72.3%	300
Credit report · Improper use	53.2%	44.0%	48.2%	300
Credit report · Incorrect information	43.3%	59.7%	50.2%	300
Credit report · Investigation problem	51.6%	48.0%	49.7%	300
Out of scope (Unknown)	82.3%	65.3%	72.9%	300

8.4 Where the model gets confused

Product-level errors are rare (9.7% of test complaints); most mistakes pick the wrong issue within the right product, which matters less for routing because departments are assigned per product.

Figure 5 · Confusion matrix, 2026 test set (rows = true class, columns = predicted)



Darker = more complaints. The diagonal is correct; the largest off-diagonal blocks sit inside the credit-reporting family and inside checking accounts.

Most frequent mistakes

True class	Predicted as	Complaints
Credit report · Improper use	Credit report · Incorrect information	95
Credit report · Investigation problem	Credit report · Incorrect information	94
Checking · Deposits & withdrawals	Checking · Managing an account	89
Credit report · Improper use	Credit report · Investigation problem	69
Checking · Managing an account	Checking · Deposits & withdrawals	60
Credit report · Incorrect information	Credit report · Investigation problem	53

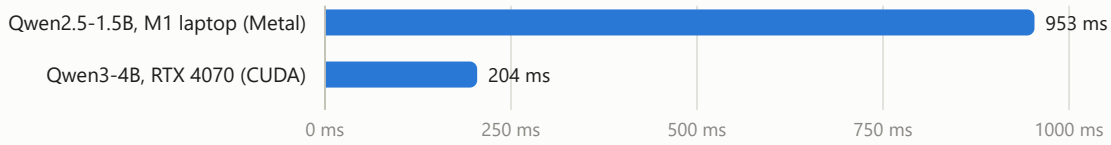
8.5 Compression check: fine-tuned adapter vs 4-bit GGUF

Same 300 test rows	10-class accuracy
Fine-tuned adapter (Transformers, 4-bit NF4 base)	64.7%
Exported Q4_K_M GGUF (llama-server, as the app runs it)	59.7%
Identical predictions between the two	81.7%

Quantizing the merged model to Q4_K_M (2.3 GiB) cost about 5 points on this sample, more than the couple of points the kit anticipated.

8.6 Speed

Figure 6 · Mean time per complaint in the deployed app



Measured over the 153 reserved complaints through each deployment's model server. Different hardware: the comparison reflects the whole setup, not the model alone. TF-IDF baseline: under 1 ms.

Measure	Value
Qwen3 GGUF, mean / median / 95th percentile (153 complaints)	204 / 201 / 270 ms
Qwen3 GGUF, mean over the 300-row export check	198 ms
Training throughput	7.67 s per step of 16 complaints (about 2.1 complaints/s)
Batch evaluation (adapter, batch 16)	about 3-8 complaints/s on the RTX 4070

9 Inspected examples

Examples are taken deterministically (sorted by complaint ID) from the 2026 test set. Narratives are public CFPB text; "XXXX" marks the CFPB's own redactions. Excerpts are shortened.

Correct predictions

Complaint excerpt	True	Predicted
"Between XXXX and XX/XX/XXXX, I discovered numerous unauthorized transactions totaling approximately {$2800.00} on my XXXX Discover credit card accounts (ending in XXXX and XXXX). These charges were made at restaurants and mercha..."	Credit card · Billing disputes	Credit card · Billing disputes
"Requesting assistance with customer service, branch escalations team, XXXX department, credit card services, and rewards department are presenting multiple challenges, because they are unaware of their own problems. It requires a ..."	Credit card · Other	Credit card · Other
"I am submitting this complaint against Navient XXXX XXXX XXXX for unfair, deceptive, and abusive acts and practices related to a private student loan that has caused substantial and ongoing financial harm despite long-term, consis..."	Out of scope (Unknown)	Out of scope (Unknown)
"Someone stole my rent check and washed the name and cashed it to themselves. {$4000.00} When I did Get notified that my rent was not paid that month I called PNC bank and they said I waited Too long to report it and would not reim..."	Checking · Managing an account	Checking · Managing an account
"Re : FCRA Dispute XXXX XXXX XXXX XXXX Correct Incomplete Data To Whom It May Concern, I am disputing the accuracy and completeness of the XXXX XXXX XXXX on my Experian credit report. Under the Fair Credit Reporting Act, Experian m..."	Credit report · Incorrect information	Credit report · Incorrect information
"I am filing a formal complaint regarding a data breach that has compromised my personal and financial information. My sensitive data may have been accessed or misused without my authorization. Under the Fair Credit Reporting Act (..."	Credit report · Improper use	Credit report · Improper use

Mistakes

Complaint excerpt	True	Predicted
"I am submitting this complaint regarding multiple inaccurate, outdated, and unverifiable accounts that I have identified on my credit report. These inaccuracies are negatively affecting my creditworthiness and financial opportunit..."	Credit report · Improper use	Credit report · Incorrect information
"On XX/XX/year> my sibling was detained and held at the XXXX XXXX XXXX. I received a phone call from an individual claiming to be a sheriff from the XXXX XXXX Sheriffs Office named Deputy XXXX Badge number XXXX. This individual sta..."	Out of scope (Unknown)	Checking · Managing an account
"Despite repeated attempts to resolve these matters, Equifax continues to allow damaging inaccuracies to persist on my credit profile. These unresolved errors are more than just frustrating they are unjust, and they are actively in..."	Credit report · Incorrect information	Credit report · Investigation problem

Complaint excerpt	True	Predicted
"I have submitted multiple formal dispute letters to TRANSUNION LLC Consumer Dispute Center . regarding inaccurate and unverifiable information on my credit report (see attached copies). Despite clear legal notice, TRANSUNION LLC..."	Credit report · Improper use	Credit report · Investigation problem
"The reported entries are inaccurate and do not reflect my true payment history. I have made several attempts to have these discrepancies reviewed and corrected, but no substantial action has been taken. Even after submitting all r..."	Credit report · Investigation problem	Credit report · Incorrect information
"Subject : Notice of Dispute, Demand for Correction, and Intent to Escalate Inaccurate Charge-Off Reporting XXXX XXXX XXXX XXXX Address] [City, State, XXXX Code] [Phone Number] [Email Address] [Date of Birth : MM/DD/YYYY] ..."	Out of scope (Unknown)	Credit report · Incorrect information
"I am writing to formally complain about my inability to access my MyWay2Go account for the past XXXX days. During this time, I have been unable to log in to the website or mobile app to view my balance or manage my account. I have..."	Checking · Deposits & withdrawals	Checking · Managing an account
"I am filing this complaint against Robinhood Financial LLC due to unfair restrictions on my ability to link debit cards to my brokerage account, inadequate customer support, failure to escalate or address internal complaints about..."	Checking · Managing an account	Checking · Deposits & withdrawals

Most mistakes are defensible readings of ambiguous narratives: a complaint about a bureau ignoring a dispute of wrong information fits both "incorrect information" and "investigation problem". This is why every prediction is confirmed by a reviewer, and corrections require a reason.

10 Application and quality assurance

Features

- **Cases**: searchable, filterable queue (product, priority, department, dates, review state), URL-backed filters and CSV export, including an export of human-confirmed labels.
- **Case detail**: full narrative, the immutable original prediction and policy assessment, review form (corrections need a reason), notes, lifecycle transitions and an append-only history.
- **Analyze and Imports**: single complaints or CSV batches go through one durable queue that survives restarts; failed rows can be retried and pending ones cancelled.
- **Model & System**: the loaded model and its training data size (read from the server), measured evaluation with limitations, storage and queue health, one-click backups.
- **Safety**: loopback-only binding, same-origin writes, revision checks against stale edits, no silent fallback to another model.

Automated checks (final state)

Check	Result
Python tests (pytest)	75 passed
Frontend tests (Vitest + Testing Library)	13 passed
Static checks	ruff: no issues · mypy: no issues in 22 files · strict TypeScript build
Layout check	Every page measured at 1,920 / 1,440 / 1,280 / 1,024 px: no overflow or misaligned controls (the app targets desktop screens, minimum width 1,000 px)
End-to-end	Complaint analysed through the browser with the Qwen3 GGUF: Credit card / Billing disputes in 0.5 s

Fixes made while moving to Windows

Problem	Fix
App crashed at start-up (WinError 32) when writing its daily backup	SQLite connections are now closed explicitly; Windows cannot rename or delete a file that is still open (macOS can)
Interface still named the Phase 1 model and 'this Mac'	The model name is read from the server's model file; wording is device-neutral
Health endpoint reported the Phase 1 dataset and re-read up to 78 MB of CSV on every poll	Reports the active model's dataset and caches row counts (about 40 ms per check)
Browser opened before the server was ready	The launcher opens the browser only after the app answers
Date filters sat 4 px below neighbouring controls	Removed inherited form spacing; all controls in the row now share one baseline
Legacy interfaces (Streamlit prototype, first React layout)	Removed; the review application is the single interface

11 Limitations

- **Labels are consumer-selected.** CFPB product and issue fields are chosen by the person filing, so some "errors" are label noise.
- **Partial epoch.** The deployed model saw 15,200 of 70,162 training complaints. It was a deliberate time trade-off; a full epoch (about 9.3 h at the measured speed) was not run.
- **Quantization loss.** The 4-bit GGUF trails the unquantized adapter by about 5 points on 300 rows.
- **Fixed taxonomy.** Three products and nine issues; everything else is "out of scope", not classified.
- **Truncation.** Only the first 2,000 characters are classified, and 25% of training complaints were shortened to 512 tokens.
- **Single operator.** No authentication or multi-user permissions; the pilot is meant for one reviewer on one machine.
- **Rules are rules.** Priority and routing come from documented phrase rules, not a trained severity model.

12 Issues encountered and how they were resolved

Issue	Resolution
PyTorch download corruption	two pip downloads of torch-2.10.0+cu126 (2.59 GB) failed SHA-256 verification (expected 4749cd32..., got c1d641e5... and 7044e20c...); a resumable curl download matched the official hash and was installed
Notebook checksum mismatch after opening it in VS Code	VS Code re-saves notebooks; the original was restored for the automated route, then training moved to the notebook itself
GPU memory full at the default batch, spilling into system memory	Batch 2 x 8 instead of 4 x 4: same effective batch, about a third faster
Low space on the system drive	Project moved to the ~950 GB D: drive; Windows enlarged its page file during training
Windows-only SQLite file locking	Explicit connection closing in storage and restore code (Section 10)

A Appendix: reproducing the results

Task	Command (from D:\SLMProject)
Start the application	`run_pilot.bat` → http://127.0.0.1:8501
Fill an empty dashboard with demo complaints	`run_demo_seed.bat`
Train (notebook)	`pc_kit\notebooks\train_qwen3_windows.ipynb` , kernel `pc_kit\.venv`
Export GGUF + 300-row check	`pc_kit\.venv\Scripts\python.exe pc_kit\src\export_gguf.py --adapter pc_kit\results\qwen3-4b-main\final_adapter --verify-rows 300`
Evaluate the GGUF on the 153 reserved complaints (app running)	`.venv\Scripts\python.exe -m scripts.evaluate_server --model-path models\complaintiq-qwen3-4b-Q4_K_M.gguf --output artifacts\qwen3_evaluation.json`
Run the tests	`.venv\Scripts\python.exe -m pytest` · `cd frontend && npm test`
Rebuild this report	`.venv\Scripts\python.exe report\build_report.py`

Evidence files

File	Contents
pc_kit/results/qwen3-4b-main/run_config.json, train_summary.json, train_log.jsonl	Exact training settings, timing and loss log
pc_kit/results/qwen3-4b-main/eval/	Test and check-set predictions, results.md / results.json
pc_kit/results/qwen3-4b-main/export/	GGUF, export manifest with SHA-256, 300-row GGUF predictions
artifacts/evaluation.json, finetuned_evaluation.json, qwen3_evaluation.json	Baseline, Qwen2.5 and Qwen3 on the 153 reserved complaints
artifacts/scope_evaluation.json, local_*_validation.json	Scope gate and Phase 1 model-selection benchmarks
pc_kit/data/manifest.json, data/finetuning/manifest.json	Dataset sources, splits, hashes and label mapping

Values observed during the session (not stored in a results file)

Item	Value
Speed at batch 4 x 4	about 11-12 s/step (first steps of the 200-step speed test, batch 4 x 4; stopped)
GPU memory at batch 4 x 4	11.9 of 12.3 GB in use (nvidia-smi); 100% busy but drawing only 121 of 200 W, i.e. waiting on memory spilled into shared system memory
Speed at batch 2 x 8	7.10 s/step (30-step speed test, batch 2 x 8)
GPU memory at batch 2 x 8	10.3 GB in use (nvidia-smi) at 182 W, i.e. fully loaded without spilling
PyTorch download	two pip downloads of torch-2.10.0+cu126 (2.59 GB) failed SHA-256 verification (expected 4749cd32..., got c1d641e5... and 7044e20c...); a resumable curl download matched the official hash and was installed

Report generated 09 October 2026, 15:19 IST from the files above.